

ABE 6031 Sensor Data Collection Final Project

Adam Camerer

2 December 2025

Materials:

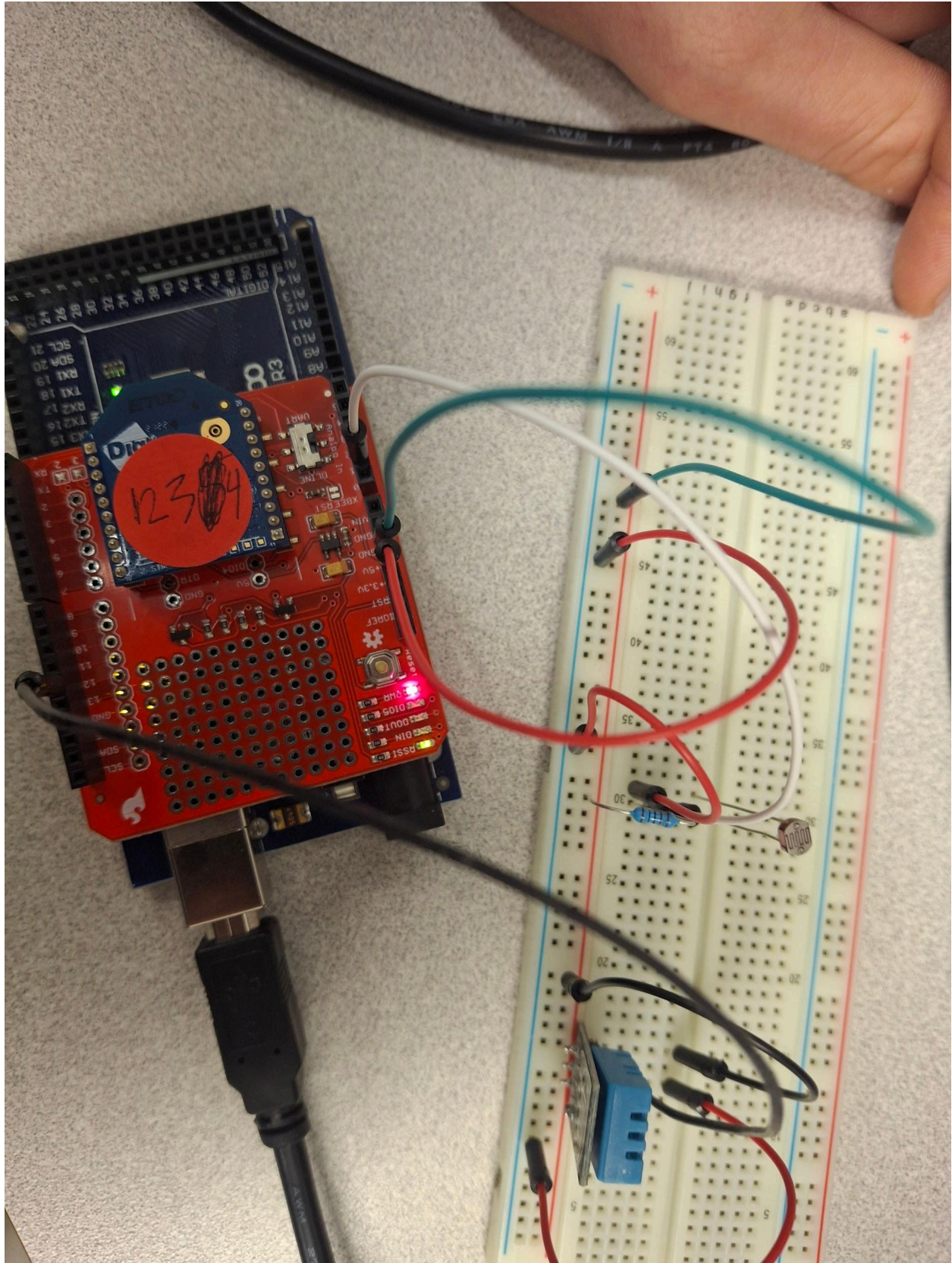
- 2 DHT 11 sensors (Temperature, Humidity)
- 1 SGP 30 sensor (TVOC)
- 2 Photoresistors
- 3 DigiKeyXBee Sensor NODes
- 1 Elegoo MEGA 2560
- 1 Arduino Uno
- 2 10K Ohm Resistors
- 2 Arduino XBee Coordinator shields
- 1 End Node XBee Shield
- 1 Raspberry Pi 4
- 1 5MP Raspberry Pi Camera
- Arduino power cables / Raspberry Pi power cables
- Jumper Wires

Methods:

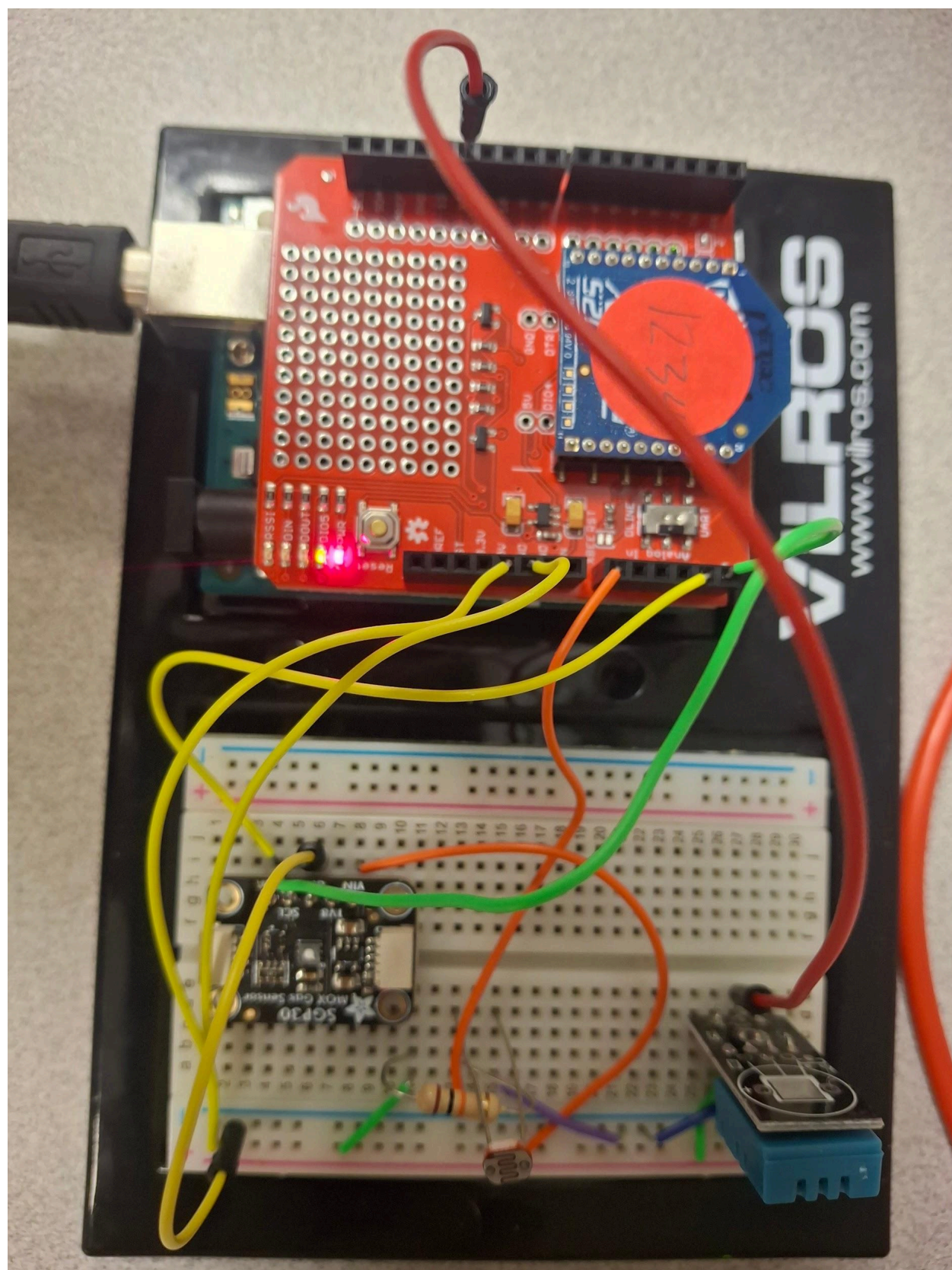
- Circuit diagram with pin connection for each sensor node

TODO: if I have more spare time

- Image of the sensor nodes



Arduino Mega Circuit



Arduino Uno Circuit

- Codes for each node

Raspberry Pi Code:

```
import serial
import time
import csv
import os
from datetime import datetime
import requests

from picamera2 import Picamera2, Preview

# =====
# SERIAL / THINGSPEAK CONFIG
# =====

SERIAL_PORT = "/dev/ttyUSB0"    # <-- change if needed (e.g. /dev/ttyACM0)
BAUD_RATE = 9600

API_KEY = ""    # <--- put your key here
CLOUD_URL = "https://api.thingspeak.com/update"

CSV_FILE = "sensornode_data.csv"

# Send to ThingSpeak at most once every 20 s
UPLOAD_INTERVAL = 20 # seconds
last_upload_time = 0

# Store latest readings for both nodes
last_n1 = {"temp": None, "hum": None, "eco2": None, "tvoc": None, "light": None}
last_n2 = {"temp": None, "hum": None, "light": None}

# =====
# CAMERA / GDRIVE CONFIG
# =====

SAVE_PATH = "/home/pi/images"    # local folder for images
GDRIVE_FOLDER = "RaspberryPiImages"    # folder name on Google Drive
CAPTURE_INTERVAL = 60    # seconds between captures

os.makedirs(SAVE_PATH, exist_ok=True)
```

```

# =====
# INIT SERIAL
# =====

ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
print("Gateway started. Listening for NODE1 and NODE2...")

# Create CSV file if it doesn't exist
try:
    with open(CSV_FILE, 'x', newline='') as f:
        writer = csv.writer(f)
        writer.writerow(["Timestamp", "Node", "Temp", "Hum", "eCO2", "TVOC", "Light"])
except FileExistsError:
    pass

def to_str_or_nan(value):
    try:
        return str(float(value))
    except:
        return "NaN"

# =====
# INIT CAMERA
# =====

picam = Picamera2()
config = picam.create_still_configuration()
picam.configure(config)

try:
    picam.start_preview(Preview.QTGL)
except Exception:
    print("Preview not supported (probably headless). Continuing without preview...")

picam.start()
print(f"Camera started. Capturing images every {CAPTURE_INTERVAL} seconds...")

next_capture_time = time.time() + CAPTURE_INTERVAL

# =====
# MAIN LOOP
# =====

try:

```

```

while True:
    now = time.time()

    # ----- 1) SERIAL / NODE HANDLING -----
    try:
        line = ser.readline().decode('utf-8').strip()
    except Exception as e:
        print(f"Serial read error: {e}")
        line = ""

    if line:
        timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        parts = [p.strip() for p in line.split(",")]

        if len(parts) >= 3:
            node_raw = parts[0]
            node = node_raw.upper()

            temp = parts[1]
            hum = parts[2]
            eco2 = None
            tvoc = None
            light = None

            if node == "NODE1":
                # Expected: NODE1,temp,hum,eco2,tvoc,light
                if len(parts) > 3:
                    eco2 = parts[3]
                if len(parts) > 4:
                    tvoc = parts[4]
                if len(parts) > 5:
                    light = parts[5]

                last_n1["temp"] = temp
                last_n1["hum"] = hum
                last_n1["eco2"] = eco2
                last_n1["tvoc"] = tvoc
                last_n1["light"] = light

            elif node == "NODE2":
                # Expected: NODE2,temp,hum,light
                if len(parts) > 3:
                    light = parts[3]

```

```

        last_n2["temp"] = temp
        last_n2["hum"] = hum
        last_n2["light"] = light

    print(f'{timestamp} | {node} | '
          f'T:{temp}C H:{hum}% eCO2:{eco2} TVOC:{tvoc} Light:{light}')

    # ---- Log to CSV ----
    with open(CSV_FILE, 'a', newline='') as f:
        writer = csv.writer(f)
        writer.writerow([timestamp, node_raw, temp, hum, eco2, tvoc, light])

    else:
        print(f'{datetime.now().strftime('%Y-%m-%d %H:%M:%S')} | Invalid data: {line}')

# ----- 2) THINGSPEAK UPLOAD (combined) -----
if now - last_upload_time >= UPLOAD_INTERVAL:
    payload = {"api_key": API_KEY}

    # NODE1 → fields 1–5
    if last_n1["temp"] is not None:
        payload["field1"] = to_str_or_nan(last_n1["temp"])
    if last_n1["hum"] is not None:
        payload["field2"] = to_str_or_nan(last_n1["hum"])
    if last_n1["eco2"] is not None:
        payload["field3"] = to_str_or_nan(last_n1["eco2"])
    if last_n1["tvoc"] is not None:
        payload["field4"] = to_str_or_nan(last_n1["tvoc"])
    if last_n1["light"] is not None:
        payload["field5"] = to_str_or_nan(last_n1["light"])

    # NODE2 → fields 6–8
    if last_n2["temp"] is not None:
        payload["field6"] = to_str_or_nan(last_n2["temp"])
    if last_n2["hum"] is not None:
        payload["field7"] = to_str_or_nan(last_n2["hum"])
    if last_n2["light"] is not None:
        payload["field8"] = to_str_or_nan(last_n2["light"])

    # Only upload if at least one field is present
    if len(payload) > 1:
        try:
            response = requests.get(CLOUD_URL, params=payload, timeout=5)
            if response.status_code == 200:

```

```

        print("→ Combined data uploaded to ThingSpeak")
    else:
        print(f"→ ThingSpeak upload failed: {response.status_code}")
    except Exception as e:
        print(f"Error sending to ThingSpeak: {e}")

    last_upload_time = now

# ----- 3) CAMERA CAPTURE / GDRIVE UPLOAD -----
if now >= next_capture_time:
    next_capture_time = now + CAPTURE_INTERVAL

    ts = time.strftime("%Y%m%d_%H%M%S")
    img_path = os.path.join(SAVE_PATH, f"image_{ts}.jpg")

    try:
        picam.capture_file(img_path)
        print(f"Saved image locally: {img_path}")
    except Exception as e:
        print(f"Error capturing image: {e}")
        continue

    try:
        print("Uploading image to Google Drive...")
        cmd = f'rclone copy \"{img_path}\" gdrive:{GDRIVE_FOLDER}/ --log-level ERROR'
        os.system(cmd)
        print(f"Uploaded to Google Drive: image_{ts}.jpg")
    except Exception as e:
        print(f"Error uploading image: {e}")

    time.sleep(0.1)

except KeyboardInterrupt:
    print("\nStopped by user.")

finally:
    try:
        picam.stop()
    except Exception:
        pass
    try:
        ser.close()
    except Exception:
        pass

```



```
print("Camera stopped, serial closed. Exiting.")
```

Arduino Uno Code:

```
#include <Wire.h>
#include "Adafruit_SGP30.h"
#include <DHT.h>

// ----- DHT CONFIG -----
#define DHTPIN 12
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

// ----- SGP30 CONFIG -----
Adafruit_SGP30 sgp;

// ----- LDR CONFIG -----
const int LDR_PIN = A0; // Photoresistor voltage divider on A0

// ----- NODE CONFIG -----
const char* NODE_ID = "NODE1"; // IMPORTANT: all caps to match Python

const unsigned long SEND_INTERVAL = 2000; // send packet every 20 s
//const unsigned long SEND_INTERVAL = 20;
unsigned long lastSendTime = 0;

// SGP30 recommended 1 s IAQ measurements
const unsigned long SGP_INTERVAL = 1000;
//const unsigned long SGP_INTERVAL = 10;
unsigned long lastSGPTime = 0;
uint16_t eco2 = 0, tvoc = 0;

// ----- SETUP -----
void setup() {
  Serial.begin(9600);
  dht.begin();
  //Serial.println("What is up");

  Wire.begin();
  Serial.println("beginning sgp");
  if (!sgp.begin()) {
    Serial.println("SGP30 not found :(");
    while (1) { delay(10); }
```

```

}
//Serial.println("Starting sgp");
sgp.IAQinit(); // start air-quality algorithm

pinMode(LDR_PIN, INPUT);

Serial.println("NODE1 (DHT11 + SGP30 + LDR) initialized...");
}

// ----- LOOP -----
void loop() {
  unsigned long now = millis();
  //Serial.println("looping");
  // Update SGP30 every 1 second
  if (now - lastSGPTime >= SGP_INTERVAL) {
    lastSGPTime = now;

    if (sgp.IAQmeasure()) {
      eco2 = sgp.eCO2; // ppm
      tvoc = sgp.TVOC; // ppb
    } else {
      Serial.println("SGP30 measurement failed");
    }
  }
}

// Send full packet every SEND_INTERVAL
if (now - lastSendTime >= SEND_INTERVAL) {
  lastSendTime = now;

  float hum = dht.readHumidity();
  float temp = dht.readTemperature();

  if (isnan(hum) || isnan(temp)) {
    Serial.println("Error reading from DHT11!");
    return;
  }

  int lightRaw = analogRead(LDR_PIN); // 0–1023

  // Format: NODE2,temp,hum,eco2,tvoc,light
  Serial.print(NODE_ID);
  Serial.print(",");
  Serial.print(temp, 1);
  Serial.print(",");

```

```

    Serial.print(hum, 1);
    Serial.print(",");
    Serial.print(eco2);
    Serial.print(",");
    Serial.print(tvoc);
    Serial.print(",");
    Serial.println(lightRaw);
  }
}

```

Arduino Mega .ino code:

```
#include <DHT.h>
```

```
// ----- DHT CONFIG -----
```

```
#define DHTPIN 12
```

```
#define DHTTYPE DHT11
```

```
DHT dht(DHTPIN, DHTTYPE);
```

```
// ----- LDR CONFIG -----
```

```
const int LDR_PIN = A0; // photoresistor on A0
```

```
// ----- NODE CONFIG -----
```

```
const char* NODE_ID = "NODE2"; // keep all caps to match Python
```

```
const unsigned long SEND_INTERVAL = 20000; // send every 20 s
```

```
unsigned long lastSendTime = 0;
```

```
// ----- SETUP -----
```

```
void setup() {
```

```
    Serial.begin(9600);
```

```
    dht.begin();
```

```
    pinMode(LDR_PIN, INPUT);
```

```
    Serial.println("NODE2 (DHT11 + LDR) initialized...");
```

```
}
```

```
// ----- LOOP -----
```

```
void loop() {
```

```
    unsigned long now = millis();
```

```
    if (now - lastSendTime >= SEND_INTERVAL) {
```

```
        lastSendTime = now;
```

```

float hum = dht.readHumidity();
float temp = dht.readTemperature();

if (isnan(hum) || isnan(temp)) {
    Serial.println("Error reading from DHT11!");
    return;
}

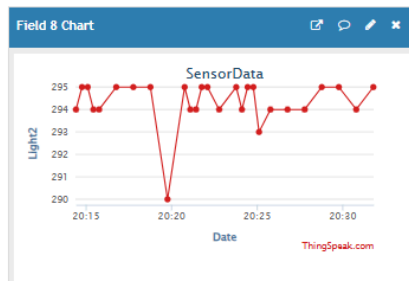
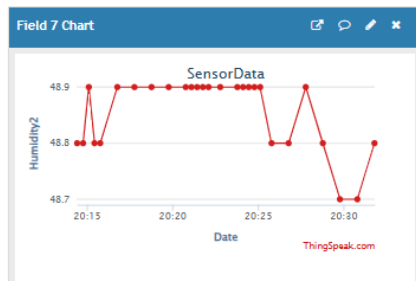
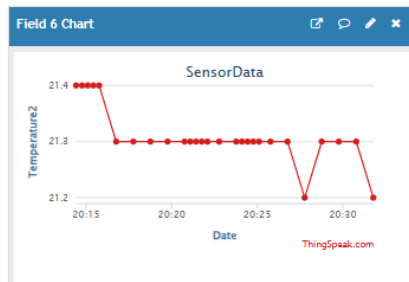
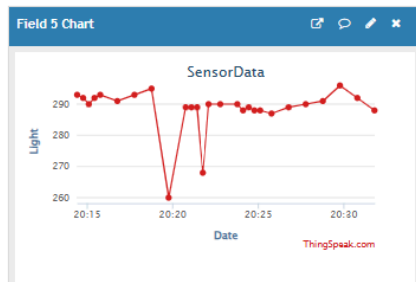
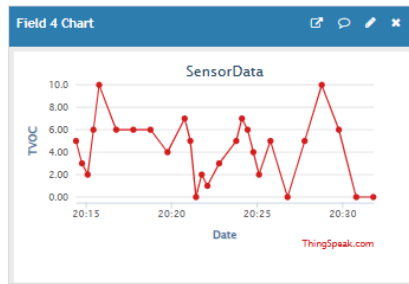
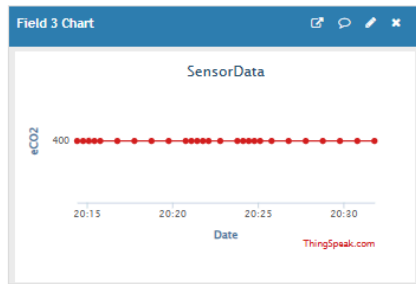
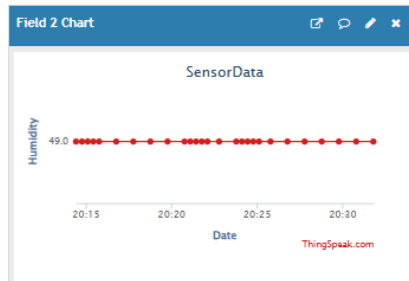
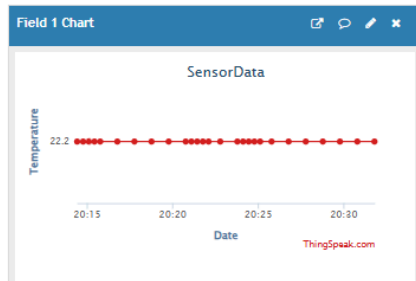
int lightRaw = analogRead(LDR_PIN); // 0–1023

// Format: NODE2,temp,hum,light
Serial.print(NODE_ID);
Serial.print(",");
Serial.print(temp, 1);
Serial.print(",");
Serial.print(hum, 1);
Serial.print(",");
Serial.println(lightRaw);
}
}

```

Results:

- Plots for each sensor data in the cloud

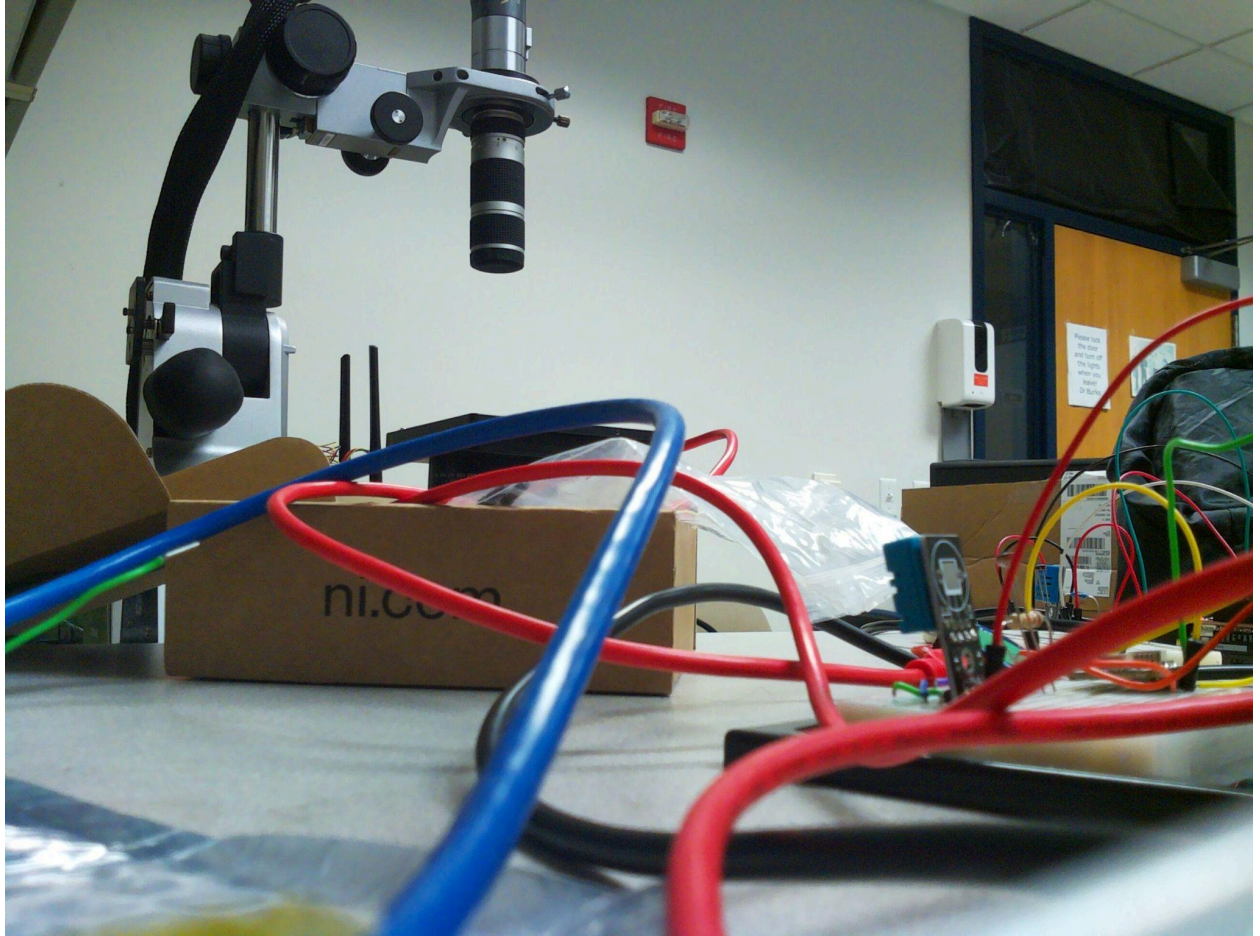


- Display some example images sent to the cloud

My Drive > RaspberryPilmages ▾

Type ▾ People ▾ Modified ▾ Source ▾

Name	Owner	Date modified 
 image_20251202_203249.jpg	 me	8:32 PM me
 image_20251202_203149.jpg	 me	8:31 PM me
 image_20251202_203049.jpg	 me	8:30 PM me
 image_20251202_202948.jpg	 me	8:29 PM me
 image_20251202_202848.jpg	 me	8:28 PM me
 image_20251202_202748.jpg	 me	8:27 PM me
 image_20251202_202648.jpg	 me	8:26 PM me
 image_20251202_202548.jpg	 me	8:25 PM me
 image_20251202_202448.jpg	 me	8:24 PM me
 image_20251202_202348.jpg	 me	8:23 PM me
 image_20251202_202248.jpg	 me	8:22 PM me
 image_20251202_202147.jpg	 me	8:21 PM me
 image_20251202_202047.jpg	 me	8:20 PM me



(Most recent image)



(Image sent 20 minutes prior)

Discussion:

- Describe the functional characteristic of each node

While each node is quite similar, the addition of the SGP30 adds a moderate bit more complexity since it detects CO₂ and TVOC, making a simple yet effective bit of complexity. The Coordinator Nodes send data wirelessly via 2.4GHz or sub GHz ISM band connection to the Raspberry Pi End Node, which essentially collects, preprocesses and send the resultant cleaned up data into the cloud for further investigation.

- What was the most interesting part of the WSN project

Honestly, this was pretty straightforward (with a lot of classmates' help), which is what I liked about it. As I already said, I think something like this would be very useful for like prototyping mass produced field data collection for smallholder farmers. I think that instrumenting with these will be useful for future personal side projects.

- How could you see improving this class activity

I think it might be cool to have these sensor nodes in portable battery powered packages we could carry around and test out on live data around the building or whatever or, even though this is a ton of work, maybe I don't know about a business competition etc of someone trying to develop a business product based off of this I guess.